Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 10

Add Your Own System Call: trace

Track B: build your own small OS (xv6)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. The Number and the User Side **2**
2.1 Registration 2
2.2 User program 2
2.3 Check 1: Unimplemented kernel call 3

3 Part 2. The Kernel Side **3**
3.1 Process mask and handler 3
3.2 Dispatch, output, and inheritance 4
3.3 Check 2: Tracing read 4

4 Part 3. Playing with the Mask **5**
4.1 All calls 5
4.2 Only fork 5
4.3 Only write 6
4.4 Only open and close 6
4.5 Observations 6

5 Conclusion **6**

1. Introduction

The aim of this laboratory work is to extend the xv6 RISC-V operating system with a new system call named `trace`. Unlike Lab 9, where `strace` observed calls from outside a Linux process, this work modifies the xv6 user interface and kernel to report selected system calls. The experiment uses system call number 23 and a per-process bit mask. Work was performed in the existing `/home/sofia/xv6-riscv` tree on Ubuntu under WSL2.

2. Part 1. The Number and the User Side

2.1. Registration

The value `SYS_trace = 23` was added after `SYS_sync = 22` in `kernel/syscall.h`. The user-level declaration was added to `user/user.h`, and `user/usys.pl` was extended so that the generated stub can issue the RISC-V `ecall` instruction.

```
// kernel/syscall.h
#define SYS_sync   22
#define SYS_trace  23

// user/user.h
int sync(void);
int trace(int);

# user/usys.pl
entry("sync");
entry("trace");
```

2.2. User program

A new `user/trace.c` accepts a mask and a command, calls `trace(mask)`, and replaces itself with the requested program using `exec()`. The program was added to the `UPROGS` list in the Makefile.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
int main(int argc, char *argv[])
{
  if(argc < 3){
```

```
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if(trace(atoi(argv[1])) < 0){
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

2.3. Check 1: Unimplemented kernel call

Before implementing the handler, xv6 booted and produced the following output.

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

This was the intended intermediate result: the user program issued system call 23, but the kernel had no registered handler yet.

3. Part 2. The Kernel Side

3.1. Process mask and handler

The `trace_mask` field was added after the process name in `struct proc` (`kernel/proc.h`). The handler in `kernel/sysproc.c` reads the integer argument and stores it in the current process.

```
// kernel/proc.h, inside struct proc
char name[16];
int trace_mask;

// kernel/sysproc.c
uint64
sys_trace(void)
{
    int mask;
    argint(0, &mask);
```

```
    myproc()->trace_mask = mask;
    return 0;
}
```

3.2. Dispatch, output, and inheritance

The handler was declared and added to the dispatcher in `kernel/syscall.c`. A name table maps call numbers to readable names. After a selected system call finishes, the kernel prints its PID, name, and return value. This xv6 version uses `printk()` rather than `printf()` for kernel output.

```
extern uint64 sys_trace(void);
// In syscalls[]:
[SYS_trace] = sys_trace,

// Immediately after p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
    printk("%d: syscall %s -> %ld\n",
           p->pid, syscallnames[num], p->trapframe->a0);

// kernel/proc.c, in kfork(), after copying the name:
np->trace_mask = p->trace_mask;
```

The inheritance line ensures that children created with `fork()` keep the parent's tracing mask.

3.3. Check 2: Tracing read

After recompiling and booting xv6, the specified command produced:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

The successful reads total $1023 + 965 + 453 = 2441$ bytes, equal to the README size stated in the laboratory sheet. The final return value of zero indicates end of file.

4. Part 3. Playing with the Mask

The mask uses bit N for system call number N. The experiments below were run in xv6; PIDs are taken from the terminal output.

4.1. All calls

The mask `2147483647` enables bits 0 through 30. The observed output was:

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

4.2. Only fork

With mask 2, the `forkforkfork` test displayed fork calls from several PIDs and finished successfully. Selected lines from the actual output:

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 5: syscall fork -> 6
6: syscall fork -> 7
7: syscall fork -> 8
7: syscall fork -> 9
...
8: syscall fork -> 66
8: syscall fork -> -1
7: syscall fork -> -1
9: syscall fork -> -1
10: syscall fork -> -1
OK
ALL TESTS PASSED
```

The negative return values indicate unsuccessful attempts once resources were exhausted in this test.

4.3. Only write

```
$ trace 65536 echo hi
hi67: syscall write -> 2

67: syscall write -> 1
```

The `echo` program made one write of two bytes for `hi` and another of one byte for the newline. The kernel logged the first call before the second write produced the newline.

4.4. Only open and close

```
$ trace 2129920 grep hello README
68: syscall open -> 3
68: syscall close -> 0
```

The exact mask is $2^{15} + 2^{21} = 32768 + 2097152 = 2129920$.

4.5. Observations

1. **Why does mask 32 show only read?** The mask 32 equals 2^5, so only bit 5 is set. Since `SYS_read` has number 5, `open` and `close` are not selected.
2. **Why do multiple PIDs print fork lines?** The line `np->trace_mask = p->trace_mask;` in `kfork()` copies the mask into every child process. This was observable in the PIDs printed by the test.
3. **Why is tracing printed between hi and the newline?** `echo` performs two `write()` calls: one for the two characters and one for the newline. The trace message is printed immediately after each call returns.
4. **Which mask selects only open and close?** $2^{15} + 2^{21} = 2129920$, as verified by the last experiment.

5. Conclusion

I extended xv6 with a user-facing system call and a kernel-side handler, connected the call number to the dispatch table, and used a per-process bitmask to control printed traces. The initial unknown-system-call result confirmed that the user-to-kernel interface was functioning; the subsequent read trace

confirmed the full implementation. The fork test demonstrated why trace settings must be inherited by child processes. Overall, this exercise connected the system-call observations from Linux Lab 9 to the implementation of the same mechanism inside xv6.